Moscow Institute of Physics and Technology
(national research university)
Phystech School of Applied Mathematics and Informatics
Chair of Discrete Mathematics

Major of study: Applied Mathematics and Informatics
Specialization: Computer Science/Информатика

FEDERATED LEARNING APPLIED TO DIABETIC RETINOPATHY PREDICTION/ПРИМЕНЕНИЕ ФЕДЕРАТИВНОГО ОБУЧЕНИЯ ДЛЯ ЗАДАЧИ ПРЕДСКАЗАНИЯ ДИАБЕТИЧЕСКОЙ РЕТИНОПАТИИ

(Bachelor)

Student:
Angulo Ramires Diego Alekhandro

_____
(student's signature)

Scientific supervisor:
Beznosikov Aleksandr Nikolaevich,
канд. физ.-мат. наук

_____
(supervisor's signature)

Adviser *(if available)***:**

_____

_____
(adviser's signature)

Moscow 2024

Moscow Institute of Physics and Technology

Phystech School of Applied Mathematics and Computer Science

Chair of Discrete Mathematics

Federated Learning applied to Diabetic Retinopathy prediction

Bachelor's thesis

Student:

Diego Alejandro Angulo Ramirez

Scientific Advisor:

Beznosikov Alexander Nikolaevich, PhD

Moscow 2024

Abstract

In this thesis, we created implementations of FedAvg and SCAFFOLD algorithms for federated learning, and used them together with two models that predict diabetic retinopathy using convolutional neural networks, from these two models one uses a ResNet architecture.
With these experiments, the idea is to simulate a real life situation with different medical centers sharing this federated model.

The results obtained, proved not only the correct work of both implementations, but the benefits of using a SCAFFOLD implementation when it comes to non-IID data, even having the best results when this is the case.

Both implementations for federated learning proved to work effectively, and outperform any independent model.
Both implementations achieve good results for IID and non-IID data, specially the ResNet18 model; which obtained up to 96% accuracy and 93% recall for predicting the presence of the sickness.
This shows how effective it can be to use dedicated architectures such as ResNet for this kind of experiments, and how federated learning implementations can work correctly and achieve outstanding results when used together with these kind of architectures.

The results obtained also suggested that for the best way to reach a faster convergence of the model we need to train only with one client at a time, this way needing a minimal communication cost for the distributed model.

Acknowledgement

I would like to dedicate a special thanks to all the people who supported me through this path of knowledge, but special thanks to...

My sister Diana, who took the place of big sister in my family while I was gone.

My brothers Matias, Fabricio and Thiago, in whom I always think when I need to smile.

My friends Rodrigo and Franco, who stood by my side during 4 of these 5 years, making this challenge a lot easier.

My university MIPT, for the opportunity of studying in the university of my dreams.

My grandparents: Maxima, Ruben, Pacifico and Irma. For constantly praying for me and supporting me.

But most importantly, to my parents Tania and Omar, who have always supported me with all their soul in my attempts to achieve greater things.

This was only possible because of you, for ever thank you.

Table of contents

Abstract i

Acknowledgment ii

Table of contents iii

List of figures v

1 Introduction 1

2 Literature Review 2
- 2.1 Artificial Intelligence and Machine Learning . . . 2
- 2.2 Deep Learning . . . 2
  - 2.2.1 Dense layers . . . 3
  - 2.2.2 Dropout layers . . . 4
  - 2.2.3 Training of Neural Network . . . 4
  - 2.2.4 Optimizers . . . 5
- 2.3 Performance Metrics . . . 5
- 2.4 Convolutional Neural Networks . . . 6
  - 2.4.1 Image Representation . . . 7
  - 2.4.2 Convolutional layers . . . 7
  - 2.4.3 Pooling layers . . . 8
  - 2.4.4 Architectures for CNNs . . . 9
- 2.5 Diabetic Retinopathy . . . 9
  - 2.5.1 Detection and Diagnosis . . . 10
- 2.6 Federated Learning . . . 12
  - 2.6.1 FedAVG . . . 12
  - 2.6.2 SCAFFOLD . . . 14
- 2.7 Related work . . . 15
  - 2.7.1 Federated Learning applied to Healthcare . . . 15
  - 2.7.2 AI models for Diabetic Retinopathy . . . 15
  - 2.7.3 Federated Learning for Diabetic Retinopathy . . . 16

3 Experiment 17
- 3.1 Architecture of CNNs used . . . 17
- 3.2 Experiment description . . . 18
- 3.3 Experiments setting . . . 19
  - 3.3.1 Libraries used . . . 19
  - 3.3.2 Dataset . . . 19
  - 3.3.3 Pre-processing of images . . . 20
  - 3.3.4 Splitting the data . . . 21
  - 3.3.5 Optimizer and Loss function . . . 21

3.3.6 Hyperparameters 22
3.3.7 Evaluation metrics 23
3.4 Experiments and Results 24
3.4.1 Experiment 1: Comparison of the federated models vs all clients 24
3.4.2 Interpretation of results for experiment 1 30
3.4.3 Final remarks about ResNet with federated learning 32
3.4.4 Experiment 2: Best value for C for efficient communication costs: . . . 32

4 Final conclusions **34**

References **35**

List of figures

1 Example structure of a Neural Network with 2 hidden layers [1] 3
2 Example of a convolution filter with multiple input and output channels [1] . . 7
3 Example of 3 different filters applied to an image [1] 8
4 Example pooling layer of a 4×4 grid [1] 8
5 Structure of ResNet-18 [2] 9
6 Typical fundus images: **(a)** *No DR*; **(b)** *Mild DR*; **(c)** *Moderate DR*; **(d)** *Severe DR*; **(e)** *Proliferative DR* [3] 11
7 Regular CNN diagram 17
8 Proliferative DR image before*(left image)* and after*(right image)* pre-processing 21
9 Losses vs Learning rates, *Left:* ResNet CNN, *Right:* Regular CNN 23
10 Performance of Regular CNN + FedAvg on IID data 24
11 Performance of Regular CNN + FedAvg on non-IID data 25
12 Performance of Regular CNN + SCAFFOLD on IID data 25
13 Performance of Regular CNN + SCAFFOLD on non-IID data 26
14 Performance of ResNet CNN + FedAvg on IID data 26
15 Performance of ResNet CNN + FedAvg on non-IID data 27
16 Performance of ResNet CNN + SCAFFOLD on IID data 27
17 Performance of ResNet CNN + SCAFFOLD on non-IID data 28
18 Results table 29
19 FedAvg and SCAFFOLD losses and accuracies after 13 rounds on ResNet CNN 31

1 Introduction

Diabetic retinopathy is a serious complication of diabetes that affects the eyes, potentially leading to blindness if is not treated. Early detection and intervention are highly important to prevent vision loss in patients with diabetes. Traditionally, diagnosing diabetic retinopathy requires personal examination of the retina or retinal images images by ophthalmologists, which is both time-consuming and prone to miss-diagnosis. With the field of AI and Deep learning models growing, it is important to acknowledge the importance these could have in medicine and the methods used for prediction of sicknesses.

Convolutional Neural Networks(CNNs), a class of deep learning algorithms, have shown remarkable success in different image recognition tasks, specially because of their ability to extract and differentiate patterns from images. In the specific case of a retina image, a CNN could easily detect different patterns that indicate the presence of diabetic retinopathy.

Furthermore, there is a big issue with only implementing CNNs for prediction of diabetic retinopathy: privacy of the data. A patient needs to authorize the use and storage of their retinal images in order to create big models that could train over this data.
This is when federated learning comes into use, with FedAvg algorithm, allowing us to train deep learning models in a distributed setting that prevents the data from being shared or stored by anyone out of the medical center where the data was collected, allowing us to create a big model without concerning about this issues.

As FedAvg algorithm for federated learning doesn't perform as good on non-IID data as it does on IID data, it is also necessary to explore SCAFFOLD algorithm, this way addressing an issue that could be present when working with different medical centers.

The approach being explored in this thesis is two implementations of CNNs, and two implementations of federated learning algorithms, this way testing with different data distributions the results and convergence of these federated learning models.

Main objectives we wish to achieve are to prove that a federated learning approach for predicting diabetic retinopathy is viable and performs better than any medical center training an algorithm on their own. Once studied the performance of federated learning algorithms, it is also important to address communication costs, and find an optimal value for amount of clients training, which is the second experiment we wish to explore.

Sections 3.1 to 3.3 of the document cover all the pre-setting of the experiments and the algorithms, and section 3.4 shows the final results of the experiments ran and present an interpretation of these mentioned.

2 Literature Review

2.1 Artificial Intelligence and Machine Learning

When we refer to classification tasks, machine learning algorithms can often be used, since with a sufficiently large amount of data it is possible to train a model to make this kind of classification task by itself, after being trained with this previously mentioned data.

Creating a computer program to perform a task can often be problematic, since we must have a fully defined algorithm that has a specific series of steps in order to give a final result. But when it comes to situations where we need to perform a task based on certain amount of data from which we can learn or learn some skill, is when we can use a machine learning algorithm, since in general AI uses techniques to learn from patterns while having a minimal human intervention.

Machine Learning implies that with enough experience in performing a task, this machine learning algorithm will perform better after an iterative learning process [4].

Machine Learning algorithms are highly applicable specially when we refer to high-dimensional data, which is our case since we'll be working with colored retina images.

We can find 3 types of learning in the Machine Learning field:

- **Supervised learning:** This kind of learning requires that we train a model with labeled data, this means that each input to the model must have also an associated correct label or output. These inputs and labels are used to improve the model during a training loop, and after that predict a label for unseen data. This kind of learning can further be subdivided into regression problems and classification problems [4].
- **Unsupervised learning:** This learning is applicable when the model must detect patterns without a label assigned to the data, the focus is to find common patterns in the data [4].
- **Reinforcement learning:** We provide the model the current state of the model, and give it a goal, letting the model learn by trial and error, trying to achieve a higher reward [4].

We will use a supervised learning model for our case, since we have labeled data already assigned to our data, data which contains a lot of recognizable patterns.

2.2 Deep Learning

Deep Learning is a sub-field of Machine Learning, using Deep Neural Networks in order to achieve results.

Neural Networks are called that way because of the way they're inspired in a biological synapses

process made by neurons, where information is transmitted with certain strength to other neurons and these new neurons send a new signal once processed all the signals received. It is important to mention that these neurons may have activation functions in between them, which will help the information to be considered only if certain threshold has been reached by a signal being sent.

Neural Networks' structure consist of an input layer, different layers called hidden layers, and a final output layer which provides results for the model. We can see a visual representation in Figure 1.

The actual learning layers are the hidden layers, which may consist of convolutions, poolings, fully connected layers and different activations.

Deep Neural Networks are specifically useful when it comes to high dimensional input data, since all the hidden layers work as a black-box which learns patterns in data and its different features. This makes Deep Neural Networks extremely effective for the purpose of analyzing the data we will use.

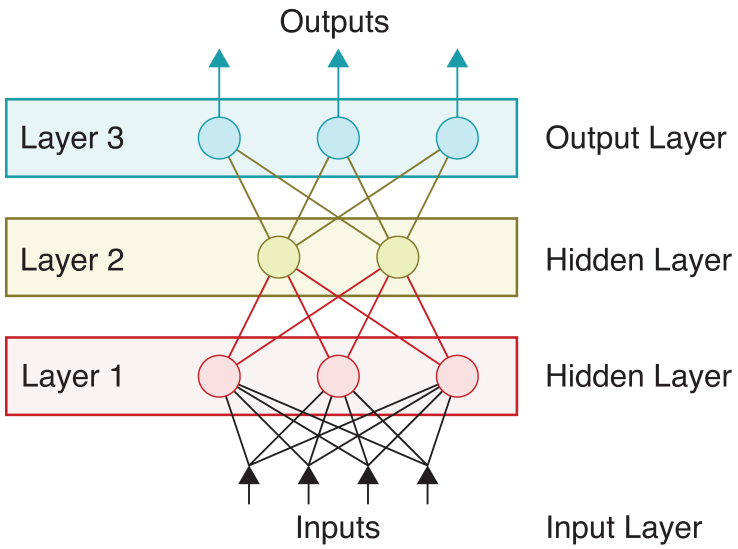


Figure 1: Example structure of a Neural Network with 2 hidden layers [1]

2.2.1 Dense layers

Dense layers are also known as fully connected layers, in this type of layer each neuron is connected to every neuron in the previous layer and is often followed by an activation function. Each connection performs a linear transformation [5].

Layers 1, 2 and 3 present in Figure 1 are examples of dense layers.

2.2.2 Dropout layers

A dropout layer is a layer used as a regularization technique; to prevent over-fitting of a model to the training data.

When the model overfits, it becomes extremely good at predicting for the training data, but doesn't really learn useful patterns to work on other data.

A dropout layer randomly '*deactivates*' a fraction of the neurons during the training of the model(by setting their output to zero), forcing the model to learn redundant patterns as the neurons on the next layer will not only depend on one neuron to detect a pattern.

During validation and testing, these layers won't '*deactivate*' any neurons[5].

2.2.3 Training of Neural Network

In order to find the parameters θ for the different connections between the neurons of a neural network, we must first of all define a Loss function $L(f(x;\theta), y)$ for which $f(x;\theta)$ is the predicted output by our neural network taking x as input and y is the expected output or true labels. The loss function is a distance metric between expected output and correct output.

To find optimal parameters, we wish to minimize this distance between predicted and true output over all data:

$$\min_{\theta} F(\theta) \triangleq \frac{1}{n} \sum_{i=1}^{n} L(f(x_i;\theta), y_i) \tag{1}$$

where n is the number of samples in our data, and y_i represents the i-th label associated with x_i input [5; 6].

To minimize this function, it is most commonly used a Gradient Descent:

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla F(\theta_t) \tag{2}$$

where θ_t are the parameters of the neural network in our t-th iteration, η_t is our learning rate or step-size and $\nabla F(\theta_t)$ is the computed gradient for all parameters of the neural network.

The gradient for all parameters is calculated by backpropagation, in general we can say backpropagation calculates the gradients by recursively applying chain rule to all the layers of the neural network starting with the gradients of the output layer with respect to $F(\theta)$ and going all the way back to the input layers [6].

Gradient descent is an optimization algorithm used to maximize or minimize an objective function, by performing several steps in the direction of the negative gradient and proportional to its magnitude.

In the deep learning sense, gradient descent will minimize the loss function defined for the model, this way improving the model's performance.

Crossentropy Loss

Crossentropy loss is a type of loss function used in multi-class classification problems. It computes the difference between the predicted probability distribution and the actual distribution of the classes from the data evaluated.

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(p_{i,c})$$

Where N is the number of samples, C is the number of classes, $y_{i,c}$ is an indicator function checking if sample i is of the class c, and $p_{i,c}$ is the predicted probability by the model that the i-th sample belongs to the class c [7].

2.2.4 Optimizers

SGD (Stochastic Gradient Descent):

SGD performs a vanilla gradient descent, with the particularity that it will perform one update iteration for each mini-batch of the whole data available for training, this way it avoids having the same gradients all the time since the mini-batch does not include the whole data [6].

2.3 Performance Metrics

Accuracy: Accuracy is the ratio of correct predictions made by the model to the total amount of samples [5].

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{1}(\hat{y}_i = y_i) \tag{3}$$

Where N is the total number of samples, $\hat{y}_i$ is the predicted label by the model, and y_i is the true label for the sample.

As our model will perform categorical classification, but 4 categories indicate the presence of the sickness in some degree, we will consider these 4 categories as only one(presence of sickness in some degree) for the purpose of using Precision, Recall and F-Score metrics.

- *True Positive(**TP**):* The image is from an eye with the disease, and the model correctly predicted it.
- *True Negative(**TN**):* The image is from a healthy eye, and the model correctly predicted it.

- *False Positive(**FP**):* The image is from a healthy eye, but the model predicted some degree of sickness.
- *False Negative(**FN**):* The image is from an eye with the disease, but the model predicted it to be healthy.

Precision: Fraction of sickness predictions by the model that were actually correct [5].

$$\text{Precision} = \frac{\text{Amount of TP}}{\text{Amount of TP} + \text{Amount of FP}} \tag{4}$$

A high precision indicates that when the model predicts the presence of the sickness, it is very likely to be correct.

Recall: Fraction of images with sickness that were correctly labeled by the model [5].

$$\text{Recall} = \frac{\text{Amount of TP}}{\text{Amount of TP} + \text{Amount of FN}} \tag{5}$$

A high recall indicates that out of all the samples labeled to have the sickness, the model is accurately predicting a large amount of them.

F1-Score: A way to combine precision and recall into one metric, taking the harmonic mean of both mentioned [1].

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times \text{Amount of TP}}{2 \times \text{Amount of TP} + \text{Amount of FP} + \text{Amount of FN}} \tag{6}$$

F1 is a very useful metric, as it gives us an idea of how the model is working on both recall and precision.

2.4 Convolutional Neural Networks

Convolutional Neural Networks(CNNs) are a type of Neural Network used mainly for working with data presented as grids, making them ideal to work with images [5].

These kind of neural networks are called this way, because of one mathematical operation used in them, called convolution. This idea of using convolutions to extract patterns was inspired by how animal's visual structure work, this way making convolutional neural networks one of the best systems to detect patterns in images [8].

A classical convolutional neural network will consist of different convolutional and pooling layers one after another, and a final few fully connected layers that will perform the final classification of the network [5].

2.4.1 Image Representation

An image is composed of a matrix of pixels, where each pixel is defined by various color channels or other distinguishing attributes.
Consequently, an image can be represented as a three-dimensional tensor. For which we can consider each channel to have assigned a 2D matrix [9].

2.4.2 Convolutional layers

A convolutional layer is the result of performing one or more convolution operations(each operation using a different filter) over all the channels in the previous layer; this resulting layer is organized into different feature maps, one feature map for each filter used [9]. Often these results are also filtered by an activation function, in order to actually detect features [5].

A convolutional layer may have a defined stride and padding, to determine how the filter will act. Stride controls the step size with which the filter will move across the input matrix. Padding acts adding zeros around the original matrix in order to apply the filter throughout the borders without problem.
Usually stride will be equal to 1, and padding equal to zero.

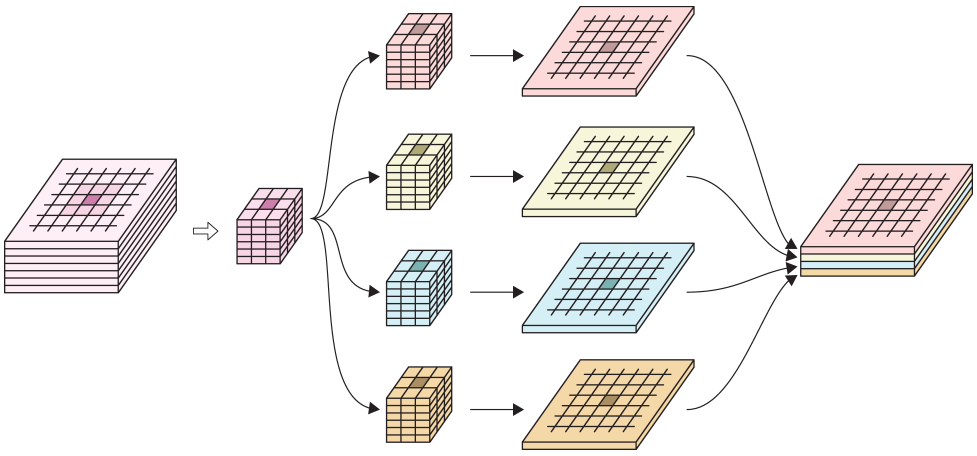


Figure 2: Example of a convolution filter with multiple input and output channels [1]

In Figure 2 we have a 3×3 filter going trough all the C_1 channels in the previous layer. For each position in the previous layer, each filter gives out a new value in one of the channels in the following layer. Once all the channels are calculated, we get a new layer with C_2 channels. For our specific image: $C_1 = 6, C_2 = 4$ (4 filters were used).

A convolution, or filter actually performs the following operation for each position (i, j)

and a $c_2 \in [0, C_2)$ filter (where C_2 is the amount of filters, or channels in the following layer).

$$s(i, j, c_2) = \sum_{c=0}^{C_1} \sum_{(k,l)\in\mathcal{N}} w(k, l, c, c_2)x(i + k, j + l, c) + b(c_2)$$

Where C_1 is the amount of input channels, $x(i, j, c)$ are the values of the activations in the previous layer, $\mathcal{N}$ is the offset for the 2D filter and $w(k, l, c, c_2), b(c_2)$ are the weights and biases for the filters. It is important to note that the weights and biases are the same for each filter, meaning that a filter will not behave differently for each position of the previous layer [9].

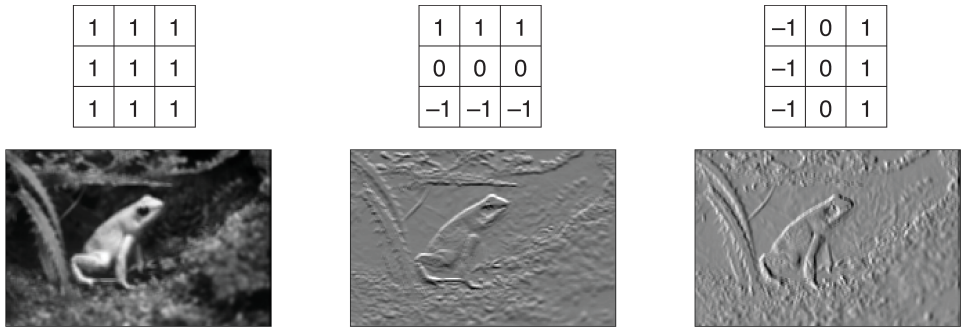


Figure 3: Example of 3 different filters applied to an image [1]

2.4.3 Pooling layers

A Pooling layer is usually present after a convolutional layer, which work as a summary of all results in a certain square region. Some pooling operations are max pooling(taking the max result inside a region) or average pooling(taking the L^2 norm for the values inside a region) among others.

A pooling layer might also help to reduce the representation size of the previous layer, but it doesn't change the amount of channels [5].

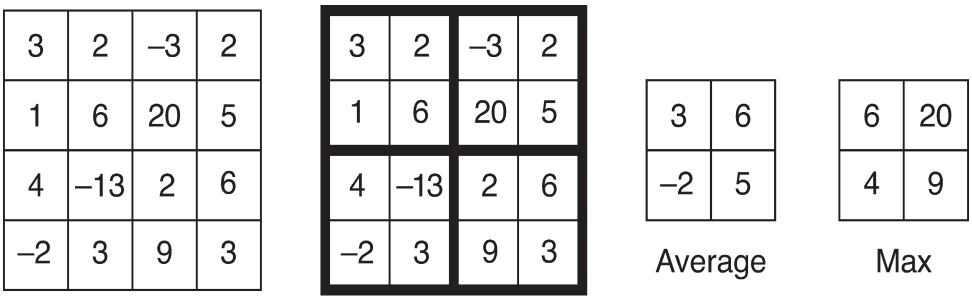


Figure 4: Example pooling layer of a 4×4 grid [1]

In Figure 4 we can see how MaxPooling and AvgPooling work.

2.4.4 Architectures for CNNs

As Deep Neural Networks were evolving and becoming better at image classification, different architectures were developed with layers more complex than just the typical network consisting of Convolutional layers, pooling layers and activations.

This architectures(such as AlexNet, ResNet, MobileNet among others), proved to perform significantly better at datasets such as ImageNet, leading to share them for being used in other vision tasks facilitating the work load and having outstanding results [9].

These architectures when used are usually called backbones, it is necessary to add a final classification layer to be used together with this backbone; this final layer is called head of the neural network.

In our case we will be testing the results of our experiment using ResNet-18 as backbone.

ResNet

ResNet, short for Residual Network, is a deep learning architecture inspired on VGG networks. ResNet introduces residual learning to deal with feature loss in the different convolutional layers. This is achieved through the use of skip connections, or shortcuts, that bypass one or more layers, allowing the model to learn residual functions with reference to the layer inputs [2].

The smallest of implementations of ResNet is ResNet-18, having only 18 layers in its structure, we can see a diagram of its structure in Figure 5.

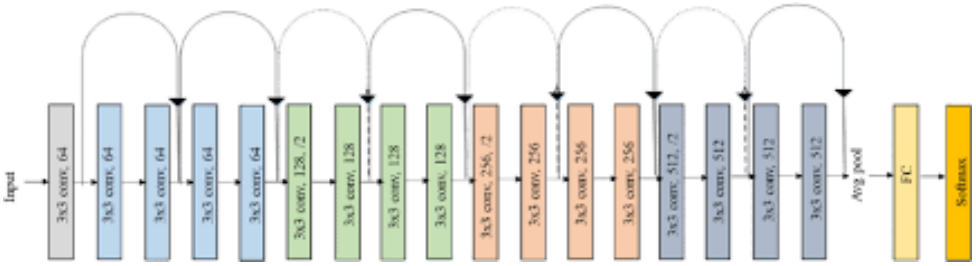


Figure 5: Structure of ResNet-18 [2]

2.5 Diabetic Retinopathy

Retinopathy refers to any disease found in the retina, Diabetic Retinopathy is the specific disease caused by diabetes of any type in a person [10].

Diabetes is a medical condition marked by high levels of glucose in the bloodstream. This being caused by a partial lack of insulin or the complete lack of it.

Diabetic retinopathy involves the pathology of capillaries, arterioles, and venules in the retina, leading to leakage or occlusion of these small vessels. Changes within the retinal capillary walls include thickening of the basement membrane, loss of pericytes, dysfunction of epithelial cells

(including loss of tight junctions), and loss of endothelial cells[11].

Retinopathy is one of the most common consequences of diabetes, with close to 30% of new cases detected having retinal lesions.

It has been shown than the longer the patient has diabetes, the higher chance or also developing retinopathy [11]. Making it extremely important to detect the presence of the disease as soon as possible in order to treat it appropriately.

As more advanced is the state of Diabetic Retinopathy in a person, they can have damages from just small haemorrhages and micro-aneurysms; which don't affect the person's sight, to fibrous tissue growth and maculopathy; causing loss of vision, blurry vision, blind spots in the vision field, in the worst of the cases leading to a complete blindness [10].

2.5.1 Detection and Diagnosis

Retinal photography using fundus cameras is one of the methods used for documenting retinopathy, these cameras can provide a view of up to a 82% of the retina and making easy to see the state of the blood vessels and other abnormalities in the retina. Thus making fundus photographs one of the main tools used to detect and diagnose Retinopathy [11].

There are specific symptoms that are visible in fundus photographs, these being:

- **Microaneurysms:** Small red dots in the retina, usually the first signs of diabetic retinopathy, the more of these microaneurysms appear in the retina can indicate a higher degree of retinopathy [10].
- **Haemorrhages:** The result of burst blood vessels in the retina, also present in red color and circular form with small irregularities in shape depending on the location of the haemorrhage. The amount of these can also indicate a higher or lower severity of diabetic retinopathy [10].

  It is important to mention that small haemorrhages can be often confused with mycroaneurysms[11].
- **Exudates:** Small white or yellow leakages of plasma with visible margins, usually present in the outer region of the retina, but when located near to the fovea(located at the center of the macula, which is located at the center of the retina) can be a cause of maculopathy or a more severe retinopathy[11; 10].
- **Cotton-wool spots:** Also referred as soft exudates, they are blurred pale spots in the retina, caused by lack of blood circulation in the region, thus leading to lack of oxygen to the nerve fibres.

  By themselves they don't cause sight problems, but can be a sign of proliferative diabetic

retinopathy [10].

- **New vessels:** Growth of new blood vessels in the retina is given due to lack of oxygen, these new vessels may present in a loop form as well, and frequently causing bleeding and obscuring vision [10].

Depending on how present are these symptoms we can categorize diabetic retinopathy in five severities: No DR, Mild DR, moderate DR, severe DR and proliferative DR.

Severity of DR	**Lesions observable in fundus image**
No DR	No lesions observable
Mild DR	Microaneurysms
Moderate DR	More lesions visible than just microaneurysms, but less lesions than in severe DR
Severe DR	Any of the following: - Intraretinal bleeding in each of 4 quadrants - Abnormal thickening in the veins of the retina - Prominent intraretinal microvascular abnormalities in 1 or more quadrants - No signs of proliferative DR
Proliferative DR	Vitreous/pre-retinal haemorrhages or new vessels surging.

Table 1: Severities of DR and the lesions present in each [12]

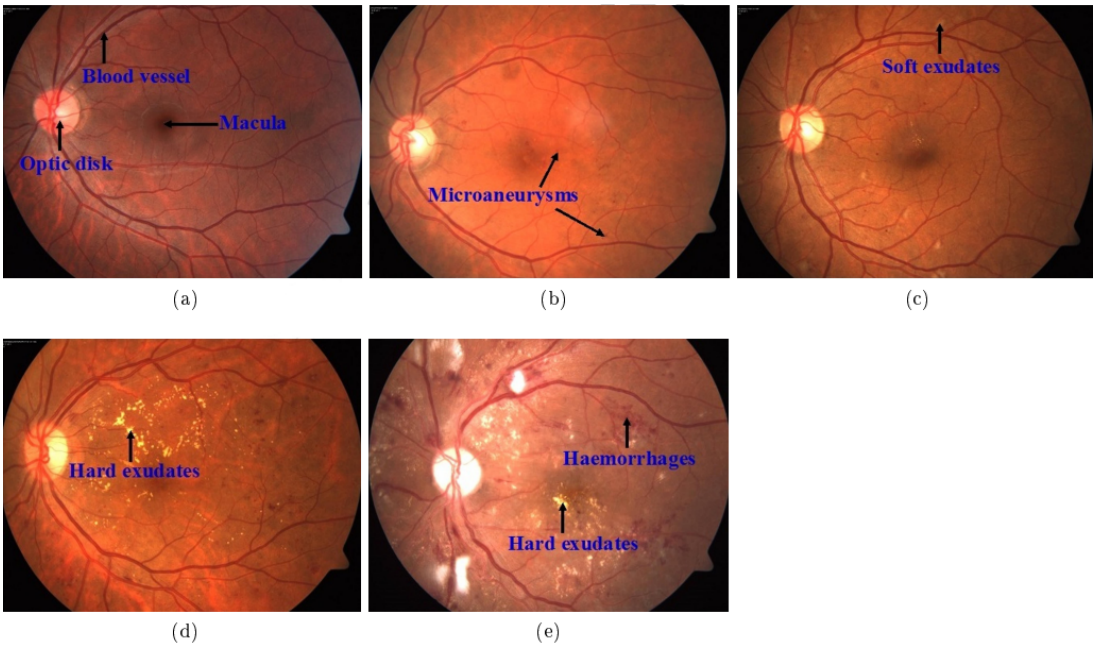


Figure 6: Typical fundus images: **(a)** *No DR*; **(b)** *Mild DR*; **(c)** *Moderate DR*; **(d)** *Severe DR*; **(e)** *Proliferative DR* [3]

As visible in Figure 6, there are visual patterns very recognizable as the sickness increases in severity, the most noticeable patterns being exudates and haemorrhages.

2.6 Federated Learning

Federated Learning is a distributed deep learning approach that allows us to train models across multiple decentralized devices or clients without the need of sharing this data to a central server. In this framework, clients (individual devices) train the model locally on their own data and then send only the updated model parameters to a central server. The server aggregates these updates to form a global model, which is then sent back to the clients for further improvement.
This process keeps going a certain number of rounds, allowing the model to learn from diverse data sources and converging to a global improved model without compromising the privacy of the data and saving in communication costs for this data[13].

The first Federated Learning algorithm(*FederatedAveraging* or *FedAvg*) and the idea of Federated Learning was proposed by Google in the paper "Communication-Efficient Learning of Deep Networks from Decentralized Data" by H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas[13].

The federated optimization problem solved with the *FedAvg* method, deals with two important properties for the distribution of the data[13]:

- **Non-IID data:** The data on a client is not always representative of the whole population, it is specific for the client.
- **Unbalanced data:** The data in some clients will be much more than in other clients, leading to clients with different amounts of data.

2.6.1 FedAVG

In this whole section we make a recap of the algorithm suggested by the paper "Communication-Efficient Learning of Deep Networks from Decentralized Data"[13].
For *FedAvg* algorithm, consider the following parameters:

n	The total number of data points across all clients.
K	Amount of clients, each client has a fixed local dataset, being $\mathcal{P}_k$ the set of indexes of the correspondent data points for client k, also $n_k = \lvert\mathcal{P}_k\rvert$.
C	Fraction of clients selected at the beginning of each round.
E	Number of epochs each client trains over its local data.
B	Size of the minibatches used for training on each client. $B = \infty$ means we use the whole local dataset.

We start by adapting eq. 1 to the federated setup, by calculating the local loss for each client:

$$\min_{\theta} F(\theta) \triangleq \sum_{k=1}^{K} \frac{n_k}{n} F_k(\theta) \text{ where } F_k(\theta) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} L(f(x_i; \theta), y_i) \tag{7}$$

This would suffice in the case of IID data across all clients by choosing $C = 1$, getting that way a full-batch gradient descent, refered as *FedSGD*.
Federated optimization improves this by adding computation and reducing communication between clients and server.

For the update of the parameters of the model(aggregation or averaging part), adapting from eq. 2:

$$\theta_{t+1} \leftarrow \theta_t - \eta \sum_{k=1}^{K} \frac{n_k}{n} \nabla F_k(\theta_t) \quad \text{where} \quad \nabla F(\theta_t) = \sum_{k=1}^{K} \frac{n_k}{n} \nabla F_k(\theta_t) \tag{8}$$

Or equivalently:

$$\forall k, \theta_{t+1}^k \leftarrow \theta_t - \eta \nabla F_k(\theta_t) \quad \text{and for the global model} \quad \theta_{t+1} \leftarrow \sum_{k=1}^{K} \frac{n_k}{n} \theta_{t+1}^k \tag{9}$$

By performing more training epochs to each client and doing more local updates to the parameters $\theta^k \leftarrow \theta - \eta \nabla F_k(\theta^k)$, we arrive at the final *FederatedAveraging* approach.

Algorithm 1 *FederatedAveraging*[13].

Server executes:
initialize θ_0
for each round $t = 1, 2, \dots$ **do**
$m \leftarrow \max(C \cdot K, 1)$
$S_t \leftarrow$ (random set of m indexes out of the K available clients)
for each client $k \in S_t$ **in parallel do**
$\theta_{t+1}^k \leftarrow \text{ClientUpdate}(k, \theta_t)$
end for
$m_t \leftarrow \sum_{k \in S_t} n_k$
$\theta_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} \theta_{t+1}^k$
end for

ClientUpdate(k, θ) :
$\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)
for each local epoch i from 1 to E **do**
for batch $b \in \mathcal{B}$ **do**
$\theta \leftarrow \theta - \eta \nabla F(\theta; b)$
end for
end for
return θ to server

The results obtained by the proposed algorithm with a simple CNN in the paper were very positive, making it a good candidate approach to implement in our case of study.

2.6.2 SCAFFOLD

In this whole section we make a recap of the algorithm suggested by the paper "SCAFFOLD: Stochastic Controlled Averaging for Federated Learning"[14].

SCAFFOLD is an algorithm which makes focus in heterogeneous data(non-IID), using variance reduction it provides a faster convergence than *FedAvg*. In general it is said that *SCAFFOLD* will perform not worst than *FedAvg*.

The main issue solved with this algorithm, is to correct a 'client-drift' introduced by FedAvg after the update in each client when the data is heterogeneous. By estimating the update direction for the server model(c) and for each client as well(c_k), it is possible to estimate the client-drift ($c - c_k$) for each client and correct it, this way obtaining a faster convergence.
For *SCAFFOLD* we consider the following parameters:

K	Amount of clients.
C	Fraction of clients selected at the beginning of each round.
E	Number of epochs each client trains over its local data.
c^t	Control variate of server after round t.
c_k^t	Control variate of client k after round t.
η_g, η_l	Global learning rate and local learning rate, correspondently.

SCAFFOLD will have control variates c, c_k for the server and for each client, initialized to 0.
In each round, the server will communicate the parameters and the control variates to all clients, each client will perform E updates in the form:

$$\theta^k \leftarrow \theta^k - \eta_l(\nabla F_k(\theta^k) + c - c_k)$$

and update the local control variate c_k: $c_k^+ \leftarrow c_k - c + \frac{1}{E\eta_l}(\theta - \theta^k)$.
And finally perform the aggregation part in the server:

$$\theta \leftarrow \theta + \frac{\eta_g}{|S|}\sum_{i\in S}(\theta^k - \theta) \quad \text{and} \quad c \leftarrow c + \frac{1}{K}\sum_{i\in S}(c_k^+ - c_k)$$

The variates work departing from the idea that an unbiased local update(without concerning about communication costs) would be: $\theta^k \leftarrow \theta^k + \frac{1}{N}\sum_{i=1}^{K}\nabla F_i(\theta^k)$, and *SCAFFOLD* uses the control variates such that: $c_i \approx \nabla F_i(\theta^k)$ and $c \approx \frac{1}{N}\sum_{i=1}^{K}\nabla F_i(\theta^k)$, this way approximating an ideal update using variates and saving on communication costs.
The following pseudocode(Algorithm 2) shows how SCAFFOLD should be implemented:

Algorithm 2 *SCAFFOLD*[14].

initialize θ, c, and c_k for all clients.
server input: global step-size η_g
client k**'s input:** local step-size η_l
for each round $t = 1, \dots$ **do**
  $m \leftarrow \max(C \cdot K, 1)$
  $S \leftarrow$ (random set of m indexes out of the K available clients)
  communicate (θ, c) to all clients $k \in S$
  for each client $k \in S$ **in parallel do**
    initialize local model $\theta^k \leftarrow \theta$
    for $e = 1, \dots, E$ **do**
      compute mini-batch gradient $\nabla F_k(\theta^k)$
      $\theta^k \leftarrow \theta^k - \eta_l \left(\nabla F_k(\theta^k) - c_k + c\right)$
    end for
    $c_k^+ \leftarrow c_k - c + \frac{1}{K\eta_l}(\theta - \theta^k)$
    communicate $(\Delta\theta^k, \Delta c_k) \leftarrow (\theta^k - \theta, c_k^+ - c_k)$
    $c_k \leftarrow c_k^+$
  end for
  $(\Delta\theta, \Delta c) \leftarrow \frac{1}{|S|} \sum_{k \in S} (\Delta\theta^k, \Delta c_k)$
  $\theta \leftarrow \theta + \eta_g \Delta\theta$ and $c \leftarrow c + \frac{|S|}{K} \Delta c$
end for

2.7 Related work

2.7.1 Federated Learning applied to Healthcare

The base idea of federated learning fits perfectly to be applied in medical institutions and used as a mean to implement different AI models to predict sickness and help with other tasks.
Given the fact that Federated Learning preserves the data at each client, it is not necessary to share sensitive data of the patients at each medical institution, this way being able to train models with large amount of data coming from different sources while preserving privacy of the patients.

Studies have already shown that Federated Learning models can perform just as good as a model with the whole data[15], without the need to share this sensitive data across clients, but sadly this is not always the case when it comes to reality.

And as of 2023, federated learning is already being successfully applied to predict different diseases, such as: COVID-19, Cancer, Pneumonia, among others[16].

But besides the advantages, it has disadvantages; like having not as good model as a model with the whole data[16] or issues related to having less reliable data in some client[15].

2.7.2 AI models for Diabetic Retinopathy

Several models have been already been created for predicting Diabetic Retinopathy, being CNN models the best ones. One of this many is the one proposed by Gangwar and Ravi in 2020[17];

using a transfer learning approach with ResNet, achieving results around 80-83% accuracy on APTOS-2019 Dataset[18], which is the Dataset we will be using as well.

Also as of 2023, other models using new innovative Neural Networks have reached an overall accuracy of 96% in the DDR Dataset[19], this model is highly accurate not only in predicting the sickness but also the exact severity of the disease.

This last mentioned model uses a method called CLAHE(Contrast Limited Adaptive Histogram Equalization) for pre-processing the images, and further for analysis implements different Convolutional layers together with high and low pass filters.

2.7.3 Federated Learning for Diabetic Retinopathy

When it comes to Federated Learning applied to these previously mentioned models, we can mention the proposed architecture by Mohan in 2023[20], implementing *FedAvg* and an innovative way to detect small lesions in the retina; achieving an accuracy of 98.6% using 5 different datasets to train(one per client), being one of them the APTOS-2019 dataset[18].

Also an implementation using *FedAvg* on IID data together with vision transformers was proposed by Chetoui and Akhloufi in 2023[21], achieving 95% accuracy on APTOS-2019 dataset[18]. In the same paper it is mentioned an implementation using CNN; achieving a 94% accuracy on the same dataset.

It is important to mention that their model only deals with predicting the presence of sickness, doesn't predict the severity of the sickness when present.

3 Experiment

All the experiments were done in a portable personal computer with a 12th Gen Intel(R) Core(TM) i7-12700H 2.70 GHz processor and a GPU NVIDIA GeForce RTX 3060 with 6GB of dedicated VRAM.

The correspondent code used for the experiments can be found at *https://github.com/diegoPenguino/Diabetic-Retinopathy*

3.1 Architecture of CNNs used

To test the efficiency and usefulness of federated learning for diabetic retinopathy prediction, we will build two different models:

The first one using a conventional and basic structure of a Convolutional Neural Network to see the correct working of FedAvg and SCAFFOLD algorithms. We will denominate this model ***Regular CNN***.

The second model will use a ResNet architecture in order to test how good results FedAvg and SCAFFOLD could obtain when implemented together with a more sophisticated CNN. We will denominate this model ***ResNet CNN***.

For the Regular CNN; the model consists of 3 blocks of convolutional layer + ReLU + MaxPooling, and two final dense layers to classify the results with a ReLU activation function and a Dropout layer between these final two.

The convolutional layers use a kernel of size 3x3 and the MaxPooling layers use a kernel of size 2x2, no padding added.

We can see a diagram of this model in Figure 7.

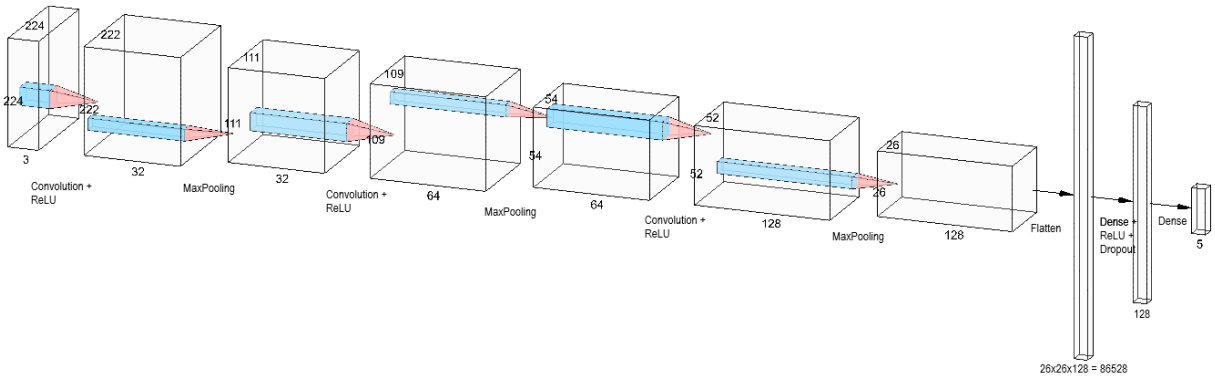


Figure 7: Regular CNN diagram

For the ResNet CNN we use the model for ResNet18 available in *torchvision* library, and added a final classification layer consisting of two dense layers, with an ReLU activation

function and a Dropout layer between them. For a diagram refer to Figure 5.
It is important to mention that ResNet18 was chosen as architecture for two main reasons:

- According to *Backbones-Review: Feature Extraction Networks for Deep Learning and Deep Reinforcement Learning Approaches*[22], ResNet performs(among others) significantly better than most architectures on Panoptic Segmentation, which is a computer vision task that implies segmenting the image in different regions or meaningful objects present in it. Which is useful for our purpose of detecting sections of the eye that present haemorrhages or exudates.
- ResNet18 has only 18 hidden layers of convolutional layers and pooling layers, but still achieves good results with this structure, using an architecture like ResNet34 or ResNet structure with more layers would make the model heavier in memory and slower, making it non-viable to perform the experiments due to the large time required and the lack of VRAM to allocate the federated model.

  As it is not planned to achieve State of the art results, ResNet18 is a good model to test our algorithms.

3.2 Experiment description

For every experiment, we simulate a real life situation:
A certain amount of medical centers wish to employ an AI model to predict diabetic retinopathy directly from fundus images. They have the opportunity to use a local model working with only their data, or use a federated learning model together with other medical centers, of course the federated learning model is more expensive, but is it really worth it to get a better performance of the model?
Once that question is answered, what is the best amount of clients that should be selected for training at each round to reach a faster convergence? Considering that each round of training will take the same time to train no matter the amount of clients training(Since they train in parallel), but the communication costs do increase when we use more clients to train the model.

For the questions asked above, we run two different experiments:

Experiment 1: Comparison of the federated models vs all clients:

For this experiment we make the federated model train for 50 rounds and selecting all clients for each round of training.
Besides, we create an amount of independent clients equivalent to the amount of clients included in the federated model, for every client there will be an independent model training with the same dataset.

Experiment 2: Best value for C:

For this experiment we make the federated learning model train for an undetermined amount of rounds, the training will stop when the model achieves certain threshold defined after the first experiment is done.

The main objective of this experiment is to check which value for C will take less rounds to reach the threshold, and as small this value can be it is better for us, since it reduces communication costs for the federated model.

For the experiments, 4 types of setting are explored:

- Regular CNN + FedAvg
- Regular CNN + SCAFFOLD
- ResNet CNN + FedAvg
- ResNet CNN + SCAFFOLD

These 4 settings will be tested on IID data and non-IID data for both experiments.

3.3 Experiments setting

3.3.1 Libraries used

The whole experiment was done using Python. Pytorch library was used for the Neural Network model[7], and Python library Matplotlib for plotting results of the model.

OpenCV computer vision library[23] was used for pre-processing the data.

3.3.2 Dataset

The dataset used for the experiment is APTOS-2019 dataset[18], available in Kaggle.

The data is labeled with one of 5 classes: No DR, Mild, Moderate, Severe and Proliferative DR. The images in the dataset are from different resolutions, they were collected from different clinics, thus taken with different fundus cameras in different lighting conditions.

The dataset consists of 3662 labeled images, distributed in the way shown in Table 2.

Severity	Amount of images
No DR	1805
Mild	370
Moderate	999
Severe	193
Proliferative DR	295

Table 2: Distribution of images

3.3.3 Pre-processing of images

In order to pre-process the images to get an improved and less noisy dataset, we applied a pre-processing algorithm adapted from Ben Graham's algorithm by Ratthachat Chatpatanasiri[24]. Being the key ideas of the pre-processing algorithm: Re-scale the image to 224x224 pixels, reduce the variation between images caused by the different conditions in which they were taken, crop the image to only contain a circular region centered around the macula of the retina and making the different patterns such as exudates or haemorrhages more noticeable.

The pre-processing consists of cropping the image and applying a Gaussian Blur effect to make patterns more noticeable.

```
import cv2
import numpy as np

def crop_image_from_gray(img, tol=7):
    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    mask = gray_img > tol
    img1 = img[:, :, 0][np.ix_(mask.any(1), mask.any(0))]
    img2 = img[:, :, 1][np.ix_(mask.any(1), mask.any(0))]
    img3 = img[:, :, 2][np.ix_(mask.any(1), mask.any(0))]
    img = np.stack([img1, img2, img3], axis=-1)
    return img

def circle_crop_v2(img):
    img = crop_image_from_gray(img)
    height, width, depth = img.shape
    largest_side = np.max((height, width))
    img = cv2.resize(img, (largest_side, largest_side))
    height, width, depth = img.shape
    x, y = int(width / 2), int(height / 2)
    r = min(x, y)
    circle_img = np.zeros((height, width), np.uint8)
    cv2.circle(circle_img, (x, y), r, 1, thickness=-1)
    img = cv2.bitwise_and(img, img, mask=circle_img)
    img = crop_image_from_gray(img)
    return img

def load_ben_color(path, IMG_SIZE, sigmaX=30):
    image = cv2.imread(path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = crop_image_from_gray(image)
```

```
    image = cv2.resize(image, (IMG_SIZE, IMG_SIZE))
    image = cv2.addWeighted(image, 4, cv2.GaussianBlur(image, (0, 0),
   sigmaX), -4, 128)
    image = circle_crop_v2(image)
    return image
```

Code 1: Pre-processing algorithm

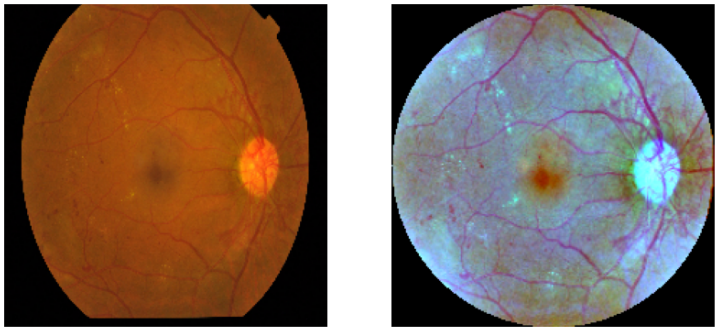


Figure 8: Proliferative DR image before*(left image)* and after*(right image)* pre-processing

3.3.4 Splitting the data

Training data is 75% of the whole dataset - 2745 images

Validation data is 10% of the whole dataset - 367 images

Test data is 15% of the whole dataset - 550 images

For the IID setting, the training data was split across 30 clients, for each class all clients have a proportional amount of images to the original distribution.

For the non-IID setting, the training data was split such that each client contains 5% of *No DR* images and a similar amount of images from one of the other classes. This was done under the assumption that this is the worst possible case for a medical center(only having one severity across all samples), but in that case it is still possible to add healthy samples to the data of this medical center.

3.3.5 Optimizer and Loss function

SGD was used as optimizer for FedAvg experiments.

For SCAFFOLD, the following optimizer was implemented based on the implementation by Junliang Li [25].

```
class Scaffold_Optimizer(torch.optim.Optimizer):
    def __init__(self, params, lr):
        defaults = dict(lr=lr)
```

```
        super(Scaffold_Optimizer, self).__init__(params, defaults)

    def step(self, server_controls, client_controls):
        for group in self.param_groups:
            for p, c, ci in zip(group["params"], server_controls.values(),
    client_controls.values()):
                dp = p.grad.data + c.data.to(device) - ci.data.to(device)
                p.data = p.data - dp.data * group["lr"]
```

Code 2: SCAFFOLD otimizer

This implementation performs the same step as SGD does, but including the control variates to the optimization step in order to achieve the intended optimization on each client for SCAFFOLD algorithm.

The loss function for the 4 experiments is Crossentropy Loss, as it is standard for multi-class classification.

3.3.6 Hyperparameters

Learning rate

For finding the best learning rates for Regular CNN and ResNet CNN settings, we run a full-dataset training loop for 40 epochs for each learning rate in the list generated and select the one with the minimal mean loss(Code 3), a visual representation of the results can be seen in Figure 9.

Note: This is the only full-dataset training we perform, and it is only done with the purpose of finding a learning rate that correctly fits the models being used for the experiments(Regular CNN and ResNet CNN).

```
init_lr, final_lr, num_iterations = 1e-1, 1e-6, 40
learning_rates = [
    init_lr * math.exp((math.log(final_lr / init_lr) / (num_iterations - 1)
    ) * i) for i in range(num_iterations)
]

mean_loss = {}
for lr in learning_rates:
    mean_loss[lr] = check_mean_loss(model, lr, epochs=20)
```

Code 3: Learning rates list

Obtaining $\eta = 0.001$ as the best learning rate for ResNet CNN and $\eta = 0.005$ as the best learning rate for Regular CNN.

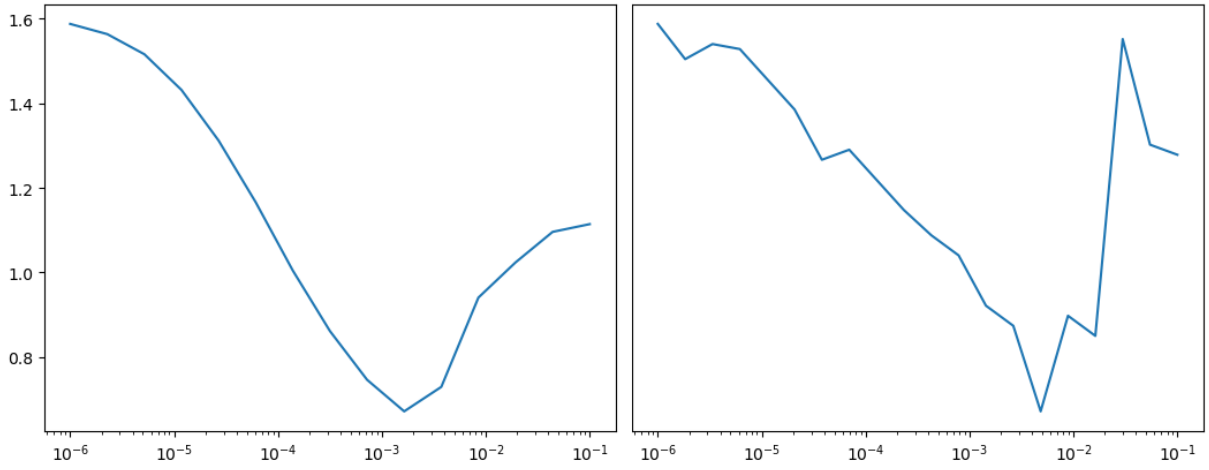


Figure 9: Losses vs Learning rates, *Left:* ResNet CNN, *Right:* Regular CNN

Hyperparameters for Federated Learning Algorithms

- $E = 5$ (training epochs per round) for all the experiments.
- $B = 16$ (Mini-batch size) for all experiments.
- Global learning rate $\eta_g = 1$ for SCAFFOLD. This value was chosen based on the implementation of SCAFFOLD by Junliang Li [25].
- Local learning rates η_l are the same values(0.001, 0.005) found for the full-dataset models.

3.3.7 Evaluation metrics

After each epoch of training on a client a validation round takes place using validation data, validation data is the one we take into account for plots and convergence of the model. This validation on validation data also takes place for the global model after each round of training is finished.

After the whole training is done, we test the models using the test data.
The following metrics were taken into consideration:

- Accuracy for multi-class prediction.
- Recall
- Precision
- F1-score.
- Binary accuracy, to check the ability of the model to detect the sickness disregarding what the severity is. Also useful to compare our results to other results obtained and mentioned in section 2.7.3.

The most important metrics we will take into account are the loss function, F1-score and Recall; in that order of importance, since the loss function is already a metric that actually tells us how

good our model is working, F1-score tells us if we're having a good precision and recall, and recall is highly important since we require a low rate of false negatives.

3.4 Experiments and Results

3.4.1 Experiment 1: Comparison of the federated models vs all clients

For all plots: the blue graph corresponds to the global model, green dotted graphs correspond to clients training, and red graphs correspond to the independent clients training only on their data.

For the global model, 5 epochs in the plots correspond to one round of training.

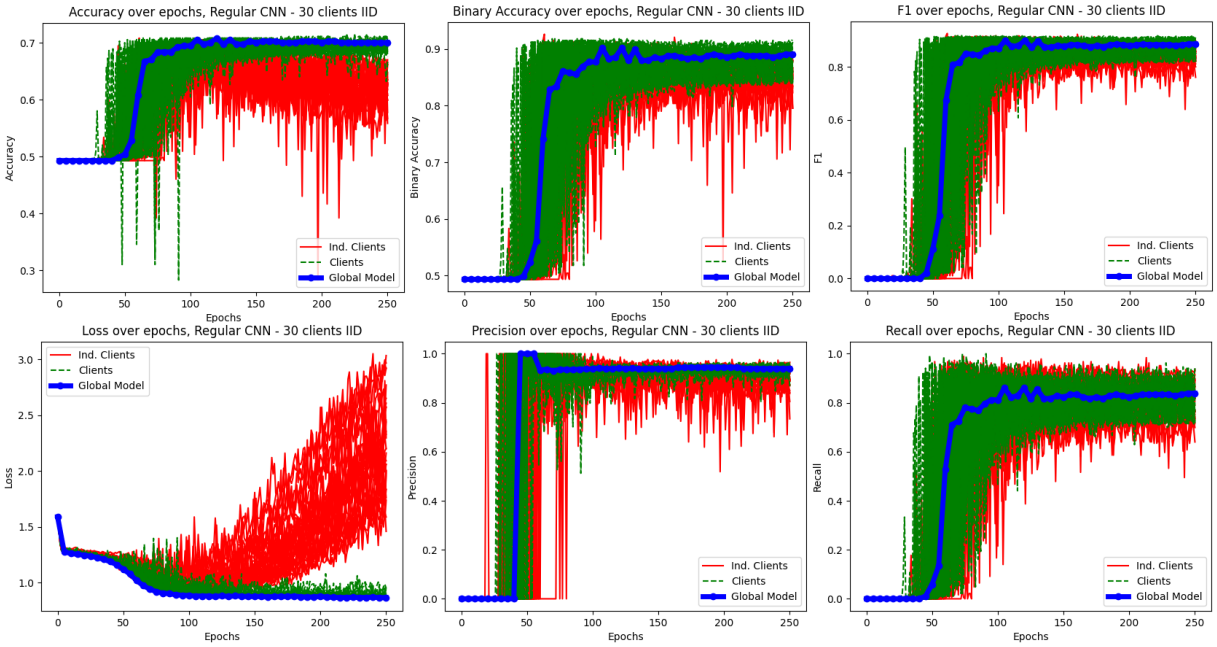


Figure 10: Performance of Regular CNN + FedAvg on IID data

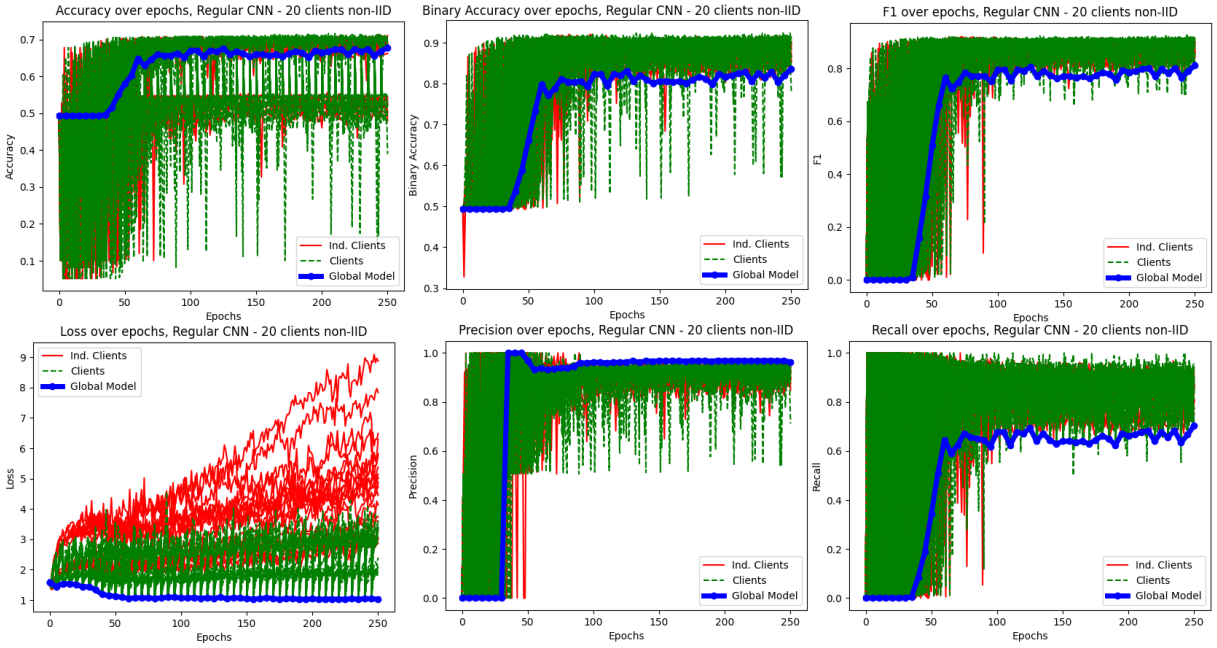


Figure 11: Performance of Regular CNN + FedAvg on non-IID data

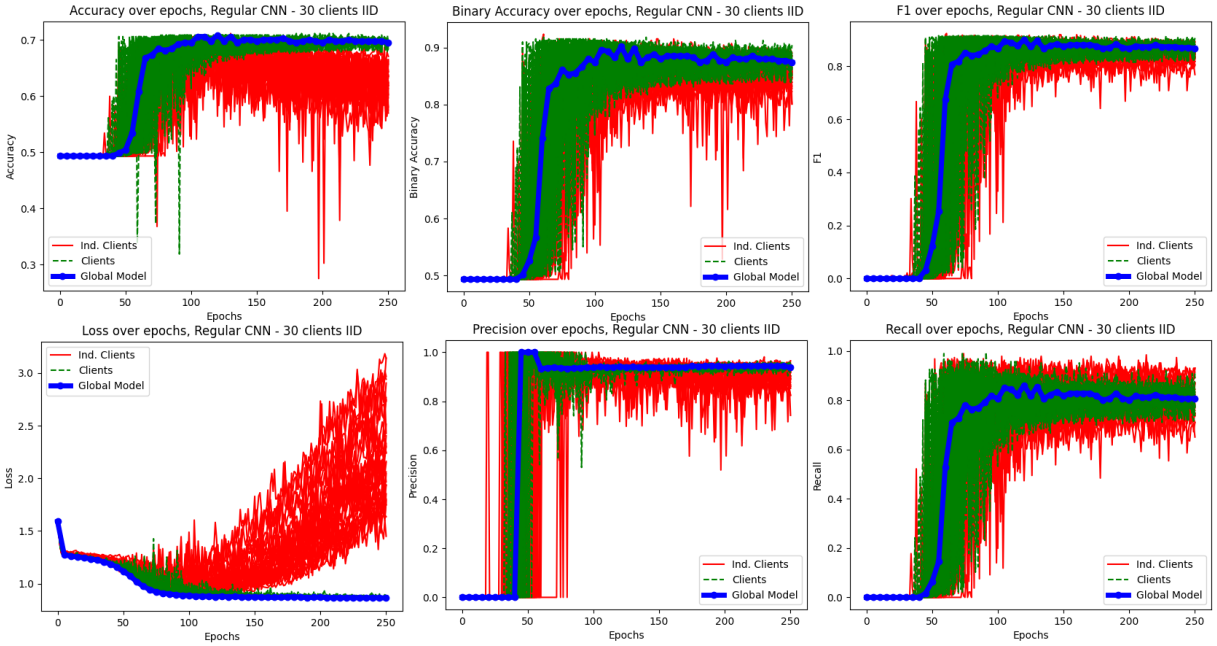


Figure 12: Performance of Regular CNN + SCAFFOLD on IID data

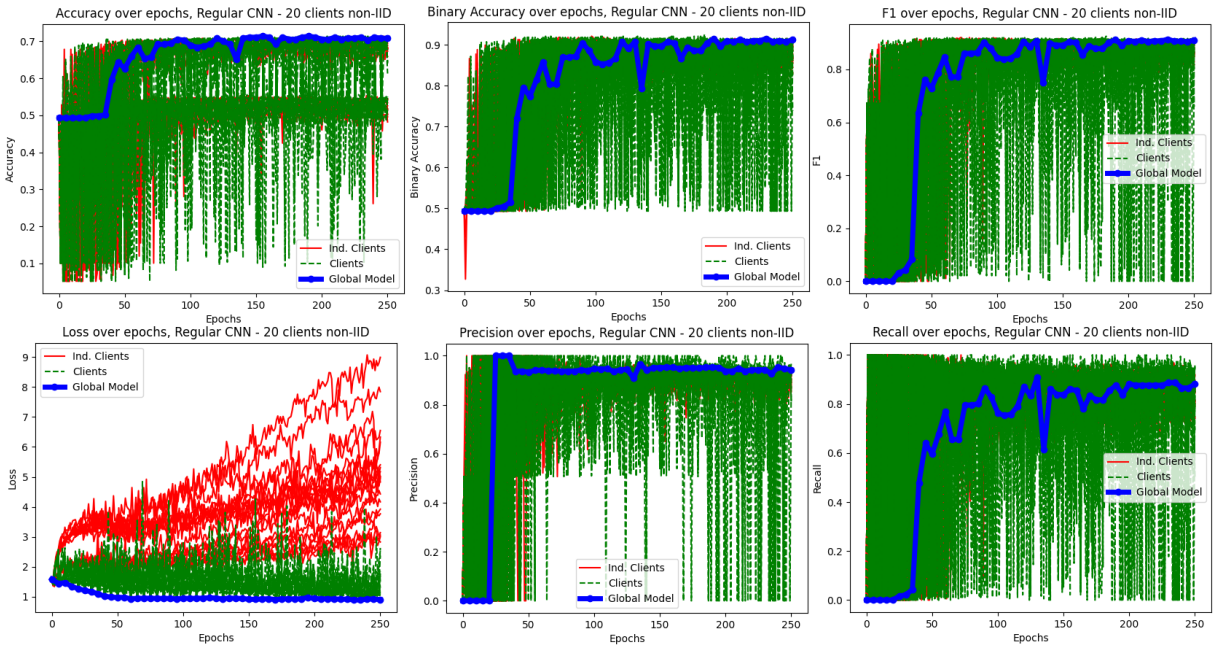


Figure 13: Performance of Regular CNN + SCAFFOLD on non-IID data

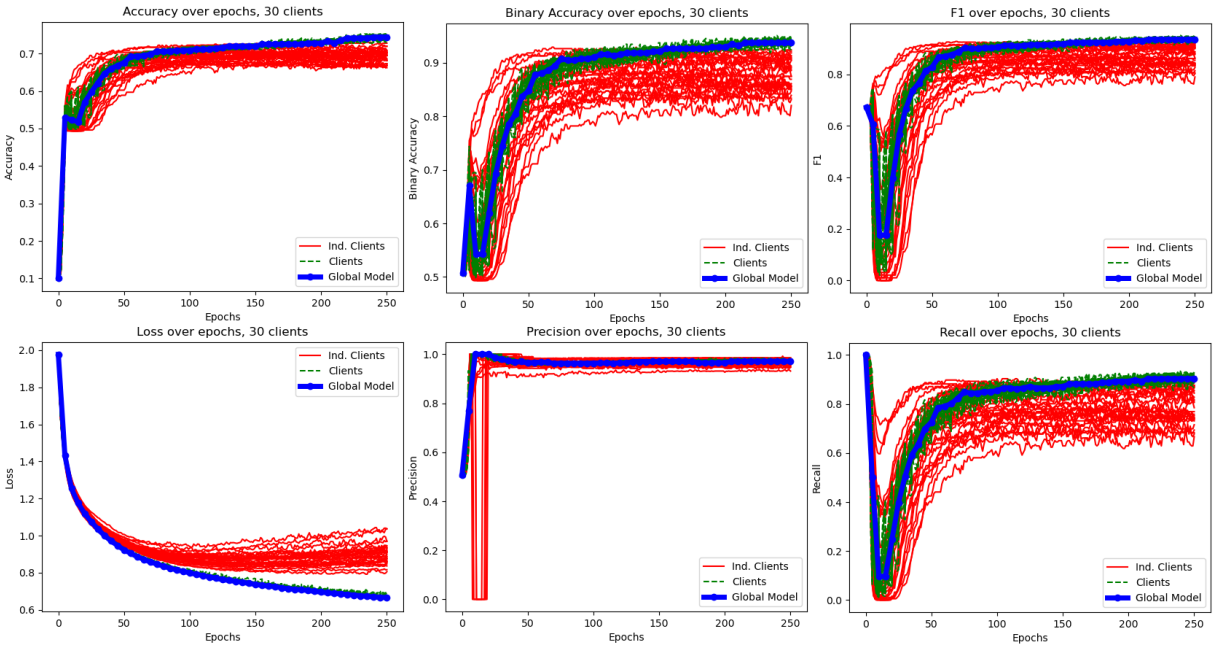


Figure 14: Performance of ResNet CNN + FedAvg on IID data

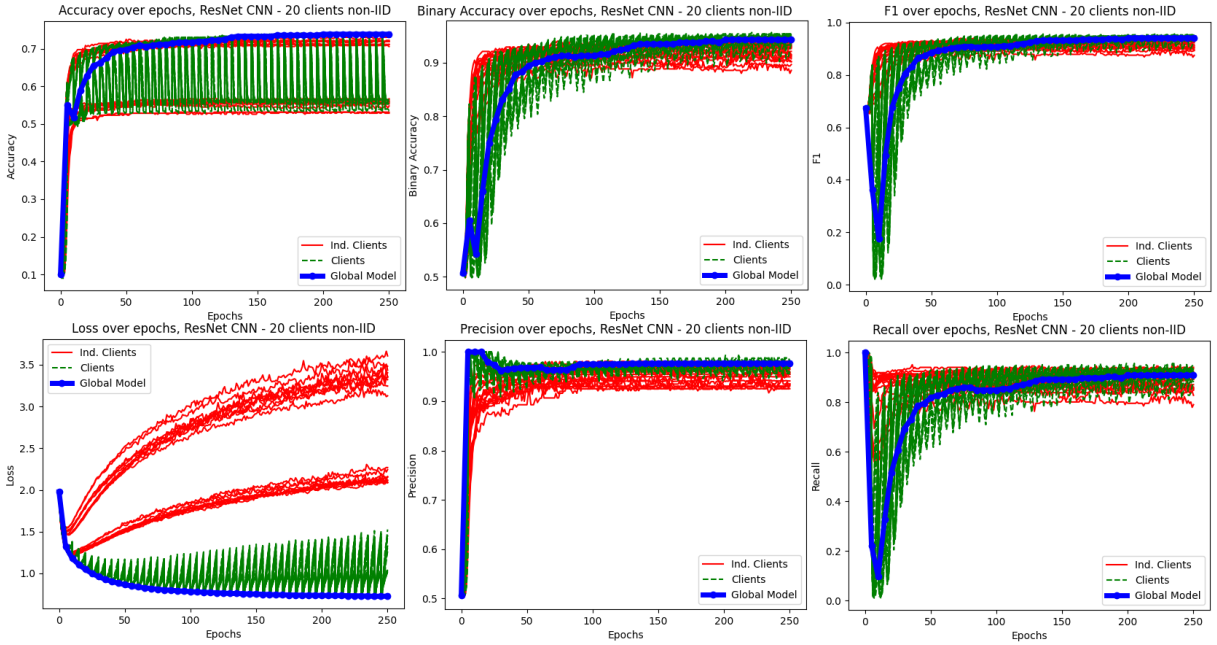


Figure 15: Performance of ResNet CNN + FedAvg on non-IID data

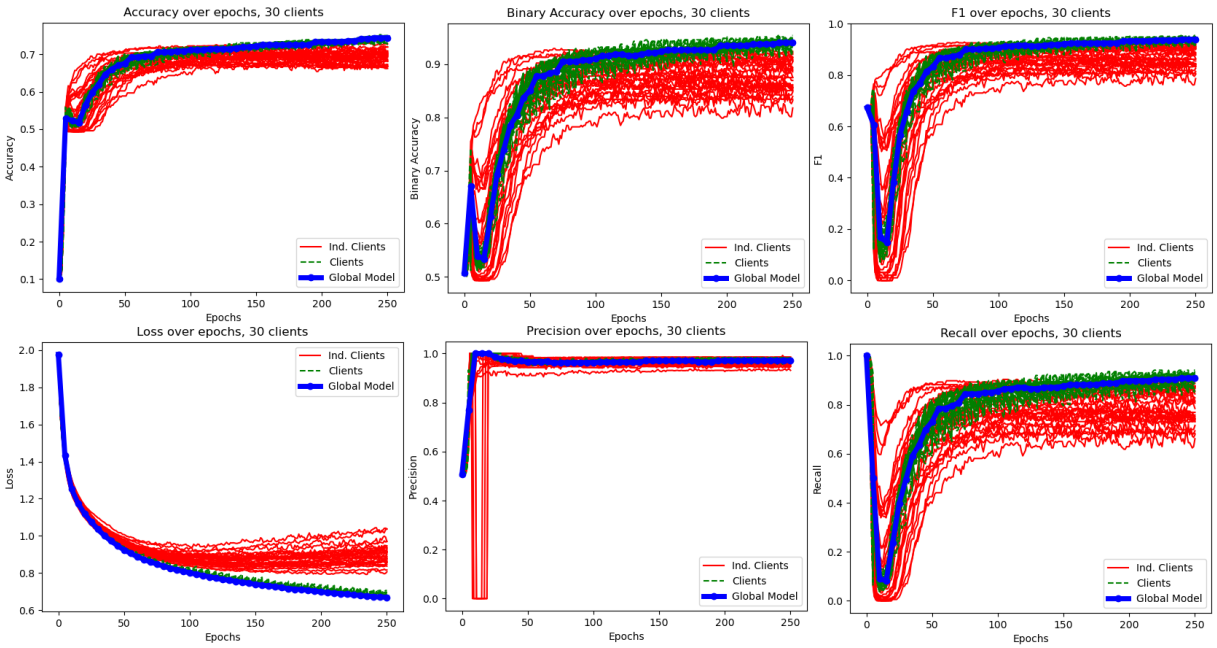


Figure 16: Performance of ResNet CNN + SCAFFOLD on IID data

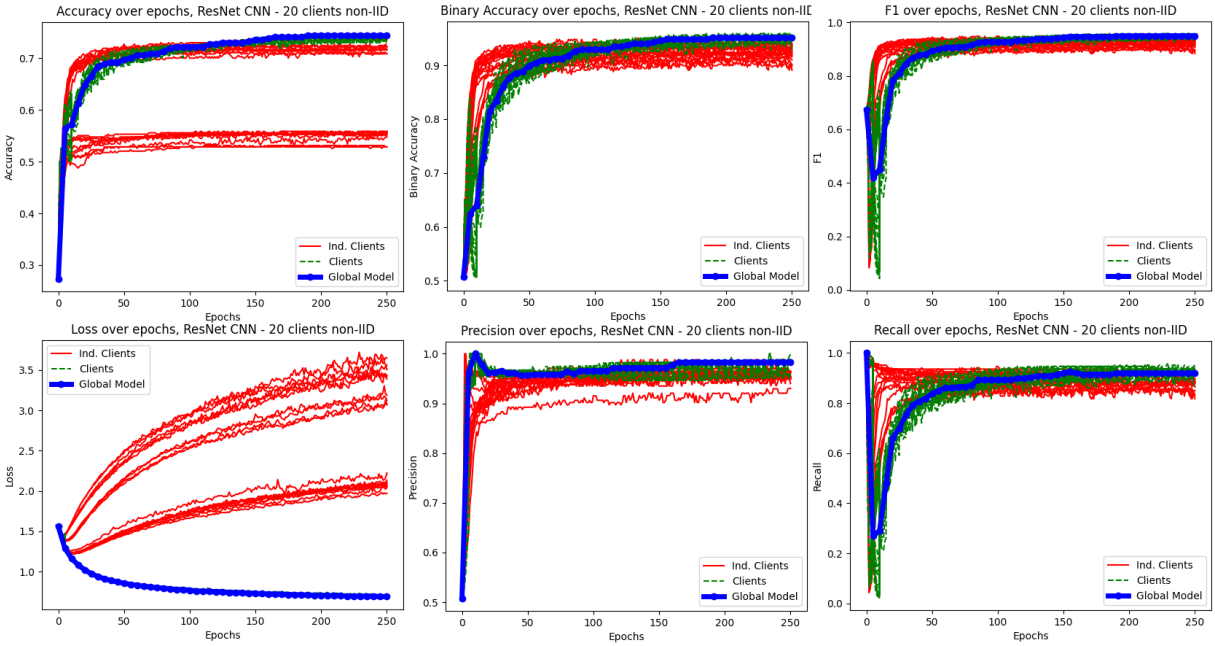


Figure 17: Performance of ResNet CNN + SCAFFOLD on non-IID data

All the final metrics of the models trained can be found in this table. For each setting; out of all the independent models it was taken the one with the minimal loss among all to include in these results.

Setting used	Data distr.	Model	Validation data						Test data					
			Loss	Accuracy	Bin. Acc.	F1	Recall	Precision	Loss	Accuracy	Bin. Acc.	F1	Recall	Precision
Regular CNN + FedAvg	IID	Federated	0.8648	70.03%	89.1%	88.64%	83.87%	93.98%	0.8201	70.36%	91.82%	91.71%	89.25%	94.32%
		Independent	1.4586	66.21%	84.74%	83.33%	75.27%	93.33%	1.4052	67.27%	87.64%	87.02%	81.72%	93.06%
	non-IID	Federated	1.0287	67.85%	83.65%	81.37%	70.43%	96.32%	0.9455	69.45%	88.36%	87.3%	78.85%	97.78%
		Independent	2.8671	68.12%	89.37%	89.54%	89.78%	89.3%	2.8771	68.18%	90.73%	91.16%	94.27%	88.26%
Regular CNN + SCAFFOLD	IID	Federated	0.8656	69.48%	87.47%	86.71%	80.65%	93.75%	0.8228	70.18%	91.64%	91.48%	88.53%	94.64%
		Independent	1.4505	66.21%	85.29%	84.3%	77.96%	91.77%	1.4105	67.64%	88.18%	87.76%	83.51%	92.46%
	non-IID	Federated	0.9038	70.84%	91.28%	91.11%	88.17%	94.25%	0.8759	68.36%	90%	90.02%	88.89%	91.18%
		Independent	2.8297	68.12%	89.37%	89.54%	89.78%	89.3%	2.8771	68.18%	90.73%	91.16%	94.27%	88.26%
ResNet CNN + FedAvg	IID	Federated	0.6655	74.39%	93.73%	93.59%	90.32%	97.11%	0.6478	76.18%	96.55%	96.5%	93.91%	99.24%
		Independent	0.8037	68.12%	87.47%	86.39%	78.49%	96.05%	0.7778	69.82%	92.91%	92.57%	87.1%	98.78%
	non-IID	Federated	0.7263	73.84%	94.28%	94.15%	90.86%	97.69%	0.6799	74.36%	96.36%	96.31%	93.55%	99.24%
		Independent	2.0837	73.3%	94.28%	94.18%	91.4%	97.14%	2.1087	73.45%	94.22%	94.21%	90.91%	97.76%
ResNet CNN + SCAFFOLD	IID	Federated	0.6681	74.39%	94.01%	93.89%	90.86%	97.13%	0.65	75.64%	96.55%	96.5%	93.91%	99.24%
		Independent	0.8039	68.12%	87.47%	86.39%	78.49%	96.05%	0.773	70%	92.91%	92.57%	87.1%	98.78%
	non-IID	Federated	0.69	74.39%	95.1%	95%	91.94%	98.28%	0.6518	74.18%	96.36%	96.31%	93.55%	99.24%
		Independent	1.9713	73.57%	94.01%	93.82%	89.78%	98.24%	1.957	74.18%	96%	95.93%	92.83%	99.23%

Figure 18: Results table

3.4.2 Interpretation of results for experiment 1

Regular CNN + FedAvg:

From Figures 10 and 11 we notice how for both types of data distribution the behaviour of the federated clients and the independent models is very erratic, but the federated model has a more steady convergence specially when it comes to the loss function.

In both types of data distribution the loss function for independent models starts to grow, specially on the non-IID case. For the IID case this is probably happening due to over-fitting of the model to the training data, probably given by the 'simple' structure of the Regular CNN model. For the non-IID case not only over-fitting is an issue, but also the distribution of the data is not helping the independent models for convergence and better results.

If we look at Figure 18, we see how FedAvg achieves better results than independent models both on validation and test data.

It is important to notice how FedAvg achieves better results with IID data, looking again at Figure 11, for non-IID data; after training round 20(100 epochs) the model did not improve its performance and stayed behind in performance compared to the IID case.

Regular CNN + SCAFFOLD

Looking at Figures 12 and 13, we notice the federated model is working much better than the independent models just as FedAvg did.

If we look at Figure 18, we see the main improvement provided by SCAFFOLD, which is a better performance for non-IID data.

Notice how the SCAFFOLD algorithm presents much better results than FedAvg for non-IID data and performance not worst than FedAvg for IID data.

ResNet CNN + FedAvg

Looking at Figures 14 and 15, we notice how the ResNet architecture is having a better behaviour than Regular CNN, specially when it comes to over-fitting of the independent models.

We can also notice how federated clients and independent models have a less erratic behaviour, probably also thanks to the ResNet architecture.

Of course for non-IID data the independent clients start to diverge given to the lack of ability to predict all the severities in the validation data.

Finally, it is important to notice how the loss for FedAvg model with IID data is still decreasing at round 50, meaning this model can still improve if run for more rounds.

A thing that is still left to improve is the behaviour of the clients in the non-IID case, since at each round of training we see how they tend to diverge from the federated model(noticeable in loss and accuracy plots).

Now looking at the final results in Figure 18, we see a significant improvement of the performance compared to the Regular CNN setting, having up to 4% better accuracy, and all the other metric above 90%(something that didn't happen with the Regular CNN setting), specially when it comes to recall.

In this setting, again we see how federated learning approach outperforms any independent model even when using a better architecture such as ResNet18.

ResNet CNN + SCAFFOLD

Theoretically this setting should outperform any other, and the results in Figures 16 and 17 show it. First of all, for the IID case we see a performance of the metrics similar or even identical to FedAvg, from which we again can say SCAFFOLD is performing not worst than FedAvg.

The differences are shown on the non-IID case, where we see how the clients for the federated model don't diverge much from the global model, helping this way to a faster convergence of the model(noticeable in loss and accuracy plots).

If we wish to see this behaviour closer we can see at Figure 19 and notice how thanks to the control variates present in SCAFFOLD help the model to diverge from the global model a lot less than FedAvg.

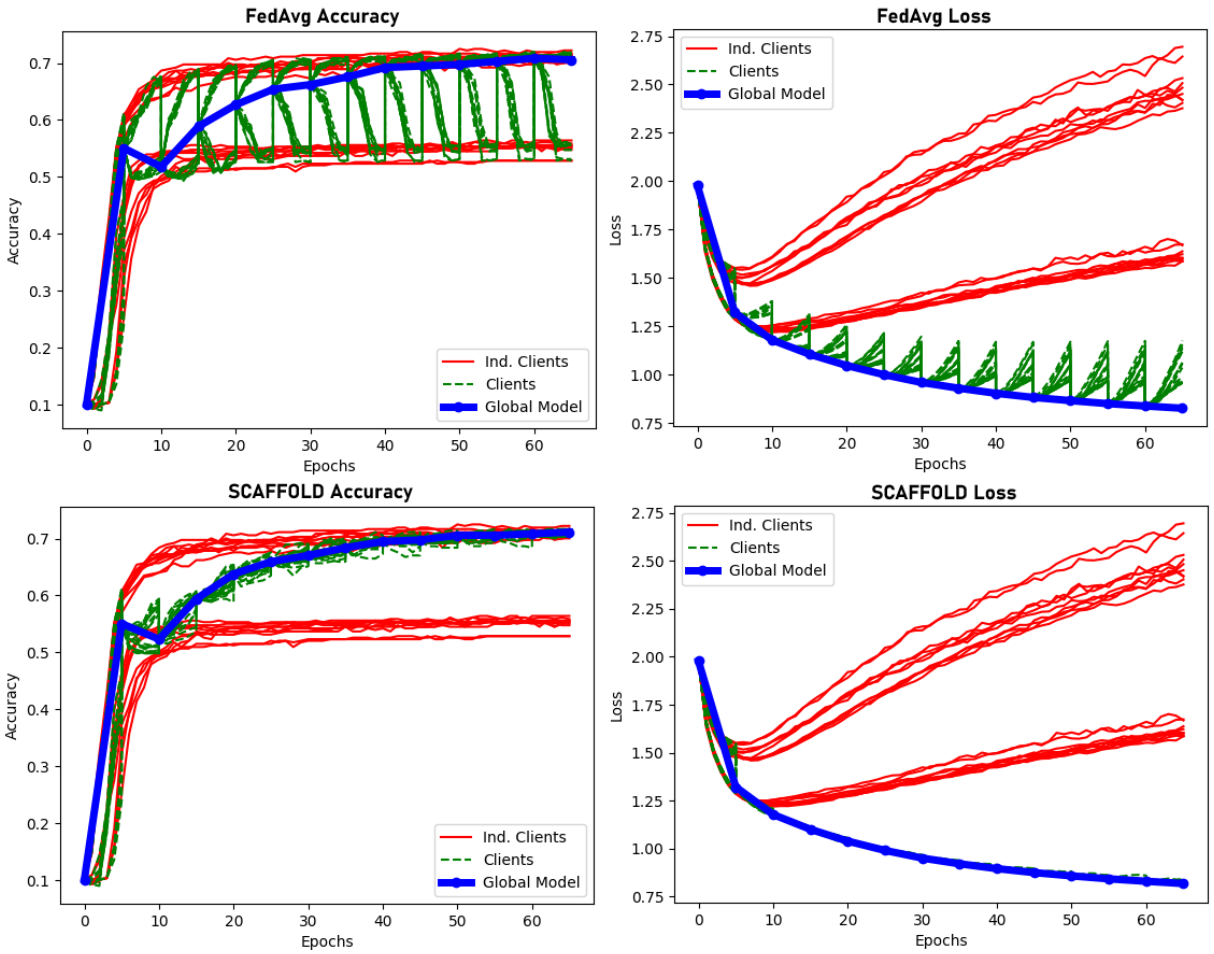


Figure 19: FedAvg and SCAFFOLD losses and accuracies after 13 rounds on ResNet CNN

Finally, looking at the results table in Figure 18, we can see how SCAFFOLD outperforms not only the independent clients, but also any other setting used for experimentation, specially when we see the results for non-IID data, we can see how the metrics for SCAFFOLD with this data distribution are much better than any other setting or model.

3.4.3 Final remarks about ResNet with federated learning

As seen in the final point of the previous interpretation of results, SCAFFOLD outperformed every other model and setting.

If we look at previous results present in section 2.7.3, ResNet together with FedAvg and SCAFFOLD provides us at least as good results as the ones obtained by Chetoui and Akhloufi on the same dataset, we can see a comparative table in Table 3.

Federated learning approach for DR	Binary accuracy on validation	Binary accuracy on test
Chetoui and Akhloufi (FedAvg + Vision Transformers)	95%	
Chetoui and Akhloufi (FedAvg + CNN)	94%	
ResNet + FedAvg	94.28%	96.36%
ResNet + SCAFFOLD	95.1%	96.36%

Table 3: Comparison to previous results

3.4.4 Experiment 2: Best value for C for efficient communication costs:

After being done with the first experiment, we set the threshold for the second experiment to be: accuracy > 0.7; recall, precision and F1 > 0.9.

As it is a threshold that is being reached by ResNet and considered to be a good model, with less than 10% change to get a false negative.

Experiment for Regular CNN

Value for C	FedAvg		SCAFFOLD	
	IID	non-IID	IID	non-IID
0.0(1 client)	84	71	72	19
0.2	>150	129	>150	66
0.5	>150	>150	>150	>150
1.0	>150	>150	>150	>150

Table 4: Values of C for all settings of Regular CNN

As the models were taking more than 24 hours to reach the threshold discussed, it was decided to stop the run and just state that the model needs more than 150 rounds to reach the threshold, or maybe it will not reach it.

We can see in Table 4 that when we train with only 1 client, we reach a faster convergence to the threshold than using more clients. This result is highly benefitial, since with a Regular CNN we can reduce the communication costs to a minimum when it comes to number of clients training each round.

SCAFFOLD again outperforms FedAvg when it comes to non-IID data, reaching the threshold much faster for $C = 0.0$ and $C = 0.2$, but still fails to reach it in less than 150 rounds for bigger values of C.

Experiment for Regular CNN

Value for C	FedAvg		SCAFFOLD	
	IID	non-IID	IID	non-IID
0.0(1 client)	29	13	21	17
0.2	40	30	38	20
0.5	40	36	44	20
1.0	43	41	44	20

Table 5: Values of C for all settings of ResNet CNN

For ResNet, the models did get to reach the threshold in all cases.
We can notice how again $C = 0.0$ is the best option for reaching this threshold faster in all cases. FedAvg and SCAFFOLD converge to the threshold almost in the same amount of rounds for IID data, but SCAFFOLD does converge in half of the time when it comes to non-IID data, again showing the good performance of SCAFFOLD for non-IID data.

With these results, we see how federated learning approach together with any of these two CNNs, will perform much better with only one client training per round, making huge saving in what comes to be communication costs.

4 Final conclusions

In this thesis, we created 4 different federated learning models and tested their performance with IID data and non-IID data.

We managed to check the efficiency and usefulness of applying Federated Learning to predict Diabetic Retinopathy, obtaining a model that can detect the sickness with high accuracy and can deal with any type of data distribution.

The Regular CNN model showed us the efficiency of federated learning even when we have a model that is prone to over-fitting.

The implementation of a ResNet18 model together with FedAvg and SCAFFOLD performed just as good as other models using visual transformers, and for sure this performance can be improved by simply using a ResNet model with more than 18 layers.

We saw how changing the number of clients training every round can lead to a faster or slower convergence of the model, and we verified that the federated learning approach can be applied with a minimal communication cost when it comes to diabetic retinopathy.

However, we also saw that our model can predict with high accuracy when the sickness is present, but it has 20% less accuracy for predicting the exact severity of the sickness, suggesting that our model may not be as good for the severity classification. This could have been caused because of the distribution of the data, and the small number of samples given for some severities.

In general, the experiments carried out for this thesis show once more the importance of AI in the present time, being applicable even when there are complications like privacy of the data or lack of resources to maintain a model with high communication costs.

It is important to remark that all the work done for this thesis is only experimental, and it is still left to implement the models studied here in a real life situation; meaning with a real server-clients architecture and data being added and labeled in real time.

For other further developments in this investigation, it could be studied how to improve the results when the data is unbalanced or mislabeled.

References

[1] Glassner Andrew. Deep learning: A visual approach. — No Starch Press, 2021.

[2] Deep residual learning for image recognition / He Kaiming, Zhang Xiangyu, Ren Shaoqing, and Sun Jian // Proceedings of the IEEE conference on computer vision and pattern recognition. — 2016. — P. 770–778.

[3] Computer-aided diagnosis of diabetic retinopathy: A review / Mookiah Muthu Rama Krishnan, Acharya U Rajendra, Chua Chua Kuang, Lim Choo Min, Ng EYK, and Laude Augustinus // Computers in biology and medicine. — 2013. — Vol. 43, no. 12. — P. 2136–2155.

[4] Janiesch Christian, Zschech Patrick, Heinrich Kai. Machine learning and deep learning // Electronic Markets. — 2021. — Vol. 31, no. 3. — P. 685–695.

[5] Goodfellow Ian, Bengio Yoshua, Courville Aaron. Deep Learning. — MIT Press, 2016. — http://www.deeplearningbook.org.

[6] Sun Ruo-Yu. Optimization for deep learning: An overview // Journal of the Operations Research Society of China. — 2020. — Vol. 8, no. 2. — P. 249–294.

[7] Pytorch: An imperative style, high-performance deep learning library / Paszke Adam, Gross Sam, Massa Francisco, Lerer Adam, Bradbury James, Chanan Gregory, Killeen Trevor, Lin Zeming, Gimelshein Natalia, Antiga Luca, et al. // Advances in neural information processing systems. — 2019. — Vol. 32.

[8] Bengio Yoshua et al. Learning deep architectures for AI // Foundations and trends® in Machine Learning. — 2009. — Vol. 2, no. 1. — P. 1–127.

[9] Szeliski Richard. Computer vision: algorithms and applications. — Springer Nature, 2022.

[10] Taylor Roy. Handbook of retinal screening in diabetes. — John Wiley & Sons, 2006.

[11] Scanlon Peter H, Sallam Ahmed, Van Wijngaarden Peter. A practical manual of diabetic retinopathy management. — John Wiley & Sons, 2017.

[12] Proposed international clinical diabetic retinopathy and diabetic macular edema disease severity scales / Wilkinson Charles P, Ferris III Frederick L, Klein Ronald E, Lee Paul P, Agardh Carl David, Davis Matthew, Dills Diana, Kampik Anselm, Pararajasegaram R, Verdaguer Juan T, et al. // Ophthalmology. — 2003. — Vol. 110, no. 9. — P. 1677–1682.

[13] Communication-efficient learning of deep networks from decentralized data / McMahan Brendan, Moore Eider, Ramage Daniel, Hampson Seth, and y Arcas Blaise Aguera // Artificial intelligence and statistics / PMLR. — 2017. — P. 1273–1282.

[14] Huang Xinmeng, Li Ping, Li Xiaoyun. Stochastic controlled averaging for federated learning with communication compression // arXiv preprint arXiv:2308.08165. — 2023.

[15] Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data / Sheller Micah J, Edwards Brandon, Reina G Anthony, Martin Jason, Pati Sarthak, Kotrotsou Aikaterini, Milchenko Mikhail, Xu Weilin, Marcus Daniel, Colen Rivka R, et al. // Scientific reports. — 2020. — Vol. 10, no. 1. — P. 12598.

[16] Sohan Md Fahimuzzman, Basalamah Anas. A systematic review on federated learning in medical image analysis // IEEE Access. — 2023.

[17] Gangwar Akhilesh Kumar, Ravi Vadlamani. Diabetic retinopathy detection using transfer learning and deep learning // Evolution in Computational Intelligence: Frontiers in Intelligent Computing: Theory and Applications (FICTA 2020), Volume 1 / Springer. — 2021. — P. 679–689.

[18] Karthik Maggie Sohier Dane. APTOS 2019 Blindness Detection. — 2019. — Access mode: https://kaggle.com/competitions/aptos2019-blindness-detection.

[19] Ashwini K, Dash Ratnakar. Grading diabetic retinopathy using multiresolution based CNN // Biomedical Signal Processing and Control. — 2023. — Vol. 86. — P. 105210.

[20] DRFL: Federated Learning in Diabetic Retinopathy Grading Using Fundus Images / Mohan N Jagan, Murugan R, Goel Tripti, and Roy Parthapratim // IEEE Transactions on Parallel and Distributed Systems. — 2023.

[21] Chetoui Mohamed, Akhloufi Moulay A. Federated Learning for Diabetic Retinopathy Detection Using Vision Transformers // BioMedInformatics. — 2023. — Vol. 3, no. 4. — P. 948–961.

[22] Backbones-review: Feature extractor networks for deep learning and deep reinforcement learning approaches in computer vision / Elharrouss Omar, Akbari Younes, Almadeed Noor, and Al-Maadeed Somaya // Computer Science Review. — 2024. — Vol. 53. — P. 100645.

[23] Bradski G. The OpenCV Library // Dr. Dobb's Journal of Software Tools. — 2000.

[24] Chatpatanasiri Ratthachat. APTOS : Eye Preprocessing in Diabetic Retinopathy // https://www.kaggle.com/. — 2019. — [Online; accessed 24-05-2024]. Access mode: https://www.kaggle.com/ratthachat/aptos-eye-preprocessing-in-diabetic-retinopathy.

[25] Li Junliang. Scaffold-Federated-Learning. — https://github.com/ki-ljl/Scaffold-Federated-Learning/. — 2022.